SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY

(SUBJECT: ARTIFICIAL NEURAL NETWORK)

(SUBJECTCODE:24SBT113)

A Mini Project on

"Weather Forecast Prediction Using ANN "

Submitted in the partial fulfillment of the requirements for the fourth semester

BACHELOR OF TECHNOLOGY
IN AIML
Submitted By,

SNEHA S	01SU24AI100
VANDANA S V	01SU24AI113
SREYA N	01SU24AI103
ALEENA V K	01SU24AI119

UNDER THE GUIDANCE OF

Prof. Mahesh Kumar V B

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

SRINIVAS UNIVERSITY INSTITUTE OF ENGINEERING & TECHNOLOGYMUKKA, MANGALURU-574146

2025-26

SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY

Department of ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

CERTIFICATE

This is to certify that SNEHA S (01SU24AI100), VANDANA S V (01SU24AI113), SREYA N (01SU24AI103), ALEENA V K (01SU24AI119) has satisfactorily completed the assessment (MINI PROJECT) in **"ARTIFICIAL NEURAL NETWORK"** prescribed by the Srinivas University for the 4st semester B. Tech course during the year **2025-26**.

MARKS AWARDED

Staff In charge

Name: Prof. Mahesh Kumar V B

Assistant Professor, Department Of AIML

TABLE OF CONTENTS

SI.no	Description	Page no.
1	Introduction	1
2	Objectives	2
3	Overview of Weather Forecast Prediction	2
4	ANN Structure for Noice Cancellation	3
5	Type of Neurons Used	4
6	Suitable Learning Law	5
7	Working Process of ANN Noice Cancellation	5
8	Advantages of ANN in Noice Cancellation	6
9	code	6
10	output	7
11	Limitations	8
12	Real-World Applications	8
13	Results and Discussion	8
14	Conclusions	8
15	Future Scope	9
16	References	9

Abstract

Weather forecast prediction is a highly critical application of data science and atmospheric physics, impacting agriculture, aviation, disaster management, and daily life. In real-world environments, meteorological patterns are highly non-linear and complex, making traditional Numerical Weather Prediction (NWP) models computationally expensive and time-consuming. Artificial Neural Networks (ANNs), implemented via Python, offer a powerful, data-driven alternative by learning historical weather patterns directly from datasets. Their ability to model complex, multi-variable relationships enables them to predict future atmospheric conditions more efficiently than conventional mathematical modeling. This project focuses on modeling an ANN-based weather prediction system using Python, mapping the network architecture, identifying suitable neuron types, and selecting an appropriate learning law. The study explains the working mechanism of the proposed model and highlights how the neural network processes historical climate data, learns from error feedback, and outputs accurate weather forecasts.

1. Introduction

In real-world environments, predicting the weather accurately is a massive challenge due to the chaotic and dynamic nature of the Earth's atmosphere. Daily activities, agricultural yields, and disaster preparedness heavily rely on understanding parameters like temperature, humidity, wind speed, and atmospheric pressure. Traditional forecasting relies on Numerical Weather Prediction (NWP), which uses complex mathematical models of the atmosphere and oceans to predict the weather. While effective, these models require supercomputers and massive computational time.

By leveraging vast datasets of historical meteorological variables—such as temperature, humidity, wind speed, and atmospheric pressure—an ANN can automatically learn and map the intricate relationships between past conditions and future weather states without relying on explicit physics equations. Once trained using robust Python machine learning libraries like Keras or Scikit-Learn, these neural networks can adapt to changing climate patterns and generate accurate forecasts with near-instantaneous inference speeds

Artificial Neural Networks (ANNs) are particularly well suited for this task because of their ability to learn complex and non-linear relationships between various meteorological features. By training on vast amounts of historical weather data using Python libraries (such as TensorFlow, Keras, or Scikit-Learn), an ANN can automatically recognize seasonal and daily patterns to predict future conditions. Unlike traditional physics-based models, neural networks can adapt quickly and provide rapid predictions once trained.

This project focuses on modeling an ANN-based noise cancellation system by analyzing and identifying the following key components:

- The network architecture used to process and transform the signal
- The types of neurons responsible for feature extraction and signal reconstruction
- The learning rule employed to train the network and minimize error

2. Objectives

The main objectives of this project are:

1. **To understand the concept of weather prediction:** Gain a clear understanding of atmospheric variables and how data-driven techniques forecast climate changes.
2. **To model an ANN architecture for weather forecasting:** Design a conceptual neural network structure that can process historical weather data inputs and generate a future forecast.
3. **To identify suitable neuron types:** Analyze the types of neurons and activation functions that are most appropriate for handling regression and classification tasks in Python.
4. **To select an appropriate learning law:** Determine the learning mechanism that allows the Python-based model to adjust its weights and minimize the difference between predicted and actual weather.
5. **To explain the working process of ANN in this application:** Describe step-by-step how the neural network receives data, processes it via Python arrays, learns from errors, and produces the final forecast.

3. Overview of Weather Forecast Prediction

Weather forecasting works by identifying patterns in historical atmospheric data and using them to estimate future conditions. The main objective is to provide reliable predictions for variables like temperature, rainfall, or storm occurrences.

There are two primary approaches to noise cancellation:

Numerical Weather Prediction (NWP)

This method relies on physical laws of fluid dynamics and thermodynamics. It involves dividing the atmosphere into a 3D grid and using supercomputers to solve complex equations. While highly accurate for long-term trends, NWP is incredibly computationally expensive and slow to process.

Data-Driven AI Forecasting (ANNs)

Artificial Neural Network-based weather prediction uses historical data (often stored in CSV files and processed via Python's Pandas library) rather than pure physics equations. The neural network learns the non-linear mappings between yesterday's weather and today's weather. This approach is highly efficient, faster at inference time, and increasingly accurate for short-to-medium-termforecasting

4. ANN Structure for Noise Cancellation

The ANN model for weather forecasting typically follows a feedforward architecture, coded in Python using libraries like Keras. Information flows from the input layer to the output layer through one or more hidden layers

4.1 Input Layer

The input layer is responsible for receiving the raw meteorological data. In a weather forecasting system, the inputs generally include:

- **Historical Data Arrays:** Features such as past temperature, humidity levels, wind speed, cloud cover, and barometric pressure.
- **Temporal Features:** Time of day, day of the year, or seasonal indicators to help the network understand cyclical weather patterns. Each neuron in the input layer represents a specific numerical feature standardized using Python's Standard Scaler. The input layer does not perform computations; it passes the scaled features to the hidden layers.

4.2 Hidden Layers

The hidden layers play a crucial role in the forecasting process by analyzing the atmospheric data and learning the relationships between different weather variables. These layers perform most of the computational work via matrix multiplications.

The main functions include:

- **Feature extraction:** Identifying important correlations (e.g., how dropping pressure relates to upcoming rain).
- **Pattern detection:** Recognizing complex, non-linear seasonal trends.
- **Data transformation:** Preparing the raw atmospheric inputs into high-level features for accurate prediction.

4.3 Output Layer

The output layer generates the final result of the network. Depending on the goal, it can take two forms:

- **Regression Output:** Predicting exact continuous values (e.g., tomorrow's temperature will be 28.5°C).
- **Classification Output:** Predicting a discrete weather state (e.g., Sunny, Rainy, Cloudy). The primary objective of the Python model during training is to minimize the error between this predicted output and the actual recorded weather data.

5. Types of Neurons Used

5.1 Linear Neurons

Linear neurons are typically used in the output layer when the task is a regression problem (e.g., predicting exact temperature or humidity percentages). These neurons provide continuous outputs that allow the network to generate precise numerical forecasts.

Linear neurons play a fundamental role in the architecture of an Artificial Neural Network designed for weather forecasting, specifically functioning within the output layer because the task involves predicting a continuous numerical value rather than producing discrete class labels. Unlike classification tasks that categorize data into predefined states—such as simply predicting if a day will be "sunny" or "rainy"—meteorological forecasting requires granular, unbounded outputs like exact temperature, wind speed, or humidity percentages. To achieve this, linear neurons provide outputs that are directly proportional to their weighted inputs, allowing the network to generate smooth and accurate numerical values that closely match the actual atmospheric measurements.

5.2 Nonlinear Neurons (ReLU / Sigmoid)

Nonlinear neurons are commonly used in the hidden layers to enable the network to learn complex atmospheric dynamics. Activation functions such as **ReLU** (Rectified Linear Unit) are highly preferred in Python implementations because they solve the vanishing gradient problem and speed up processing. If the model is predicting a categorical weather condition (like "Rain" vs. "No Rain"), a **Sigmoid** or **Softmax** neuron is used in the output layer.

Nonlinear neurons are fundamental components within the hidden layers of an Artificial Neural Network, enabling the model to capture and map the highly complex, non-linear relationships inherent in atmospheric data. Without these specific activation functions, the network would essentially operate as a simple linear regression model, incapable of deciphering the chaotic interactions between variables like humidity, wind speed, and barometric pressure Alternatively, for multi-class categorical predictions, such as forecasting whether tomorrow will be "Sunny," "Cloudy," or "Stormy," the Softmax activation function is employed in the output layer to generate a normalized probability distribution across all possible weather states, ensuring the system can fluidly adapt to both continuous regression and discrete meteorological classification tasks.

5.3 Adaptive Neurons

Adaptive neurons adjust their weights dynamically during training based on the input data and calculated error. This adaptability helps the network respond effectively to changing noise conditions and improves its ability to generalize across different environments. As a result, the system becomes more robust and capable of handling real-world noise variations.

6. Suitable Learning Law

The most suitable learning law for weather forecast prediction is Error-Correction Learning (specifically Backpropagation), which falls under supervised learning. In this Python-based approach, the network is trained using pairs of historical inputs (e.g., Monday's weather) and target outputs (e.g., Tuesday's weather).

The most suitable learning law for training an Artificial Neural Network in weather forecast prediction is Error-Correction Learning, which operates under the paradigm of supervised learning. In this data-driven approach, the neural network is systematically trained using paired datasets of historical atmospheric inputs—such as humidity, pressure, and wind speed—alongside their corresponding target outputs, like the subsequent day's actual temperature. During the training phase, the model generates a preliminary forecast and continuously compares its predicted output against the actual recorded weather data to compute a quantifiable error.

By leveraging advanced gradient-based optimization algorithms —such as the Adam optimizer commonly implemented in Python libraries like Keras—the network utilizes backpropagation to iteratively adjust its internal interconnected weights and biases. This dynamic process of error-correction progressively minimizes the MSE over multiple training epochs, ultimately refining the network's ability to accurately map complex meteorological relationships and predict real-world weather patterns with high precision.

Why Error-Correction Learning?

- The network learns from thousands of rows of historical CSV data.
- By comparing its predicted weather to the actual historical weather, the network identifies how far off its forecast was.
- This difference (the error) is backpropagated through the Python model to adjust weights.

7. Working Process of ANN Weather Prediction

1. Historical data is fed into the network: The input layer receives data frames (processed via Python) containing temperature, humidity, and pressure values.
2. Hidden layers analyze atmospheric patterns: The hidden layers process the input using weighted connections and ReLU activation functions to identify correlations between variables.
3. Output layer generates a forecast: After processing, the output layer predicts the future condition (e.g., tomorrow's temperature).
4. Error between predicted and actual weather is computed: The predicted output is compared with the actual historical data to calculate the MSE.
5. Weights are updated to reduce error: Using Backpropagation and Python optimizers (like Adam), the network adjusts its weights and bias values.
6. Process repeats until error is minimized: Training continues for multiple epochs until the network effectively learns the climate patterns and produces accurate forecasts.

8. Advantages of ANN in Noise Cancellation

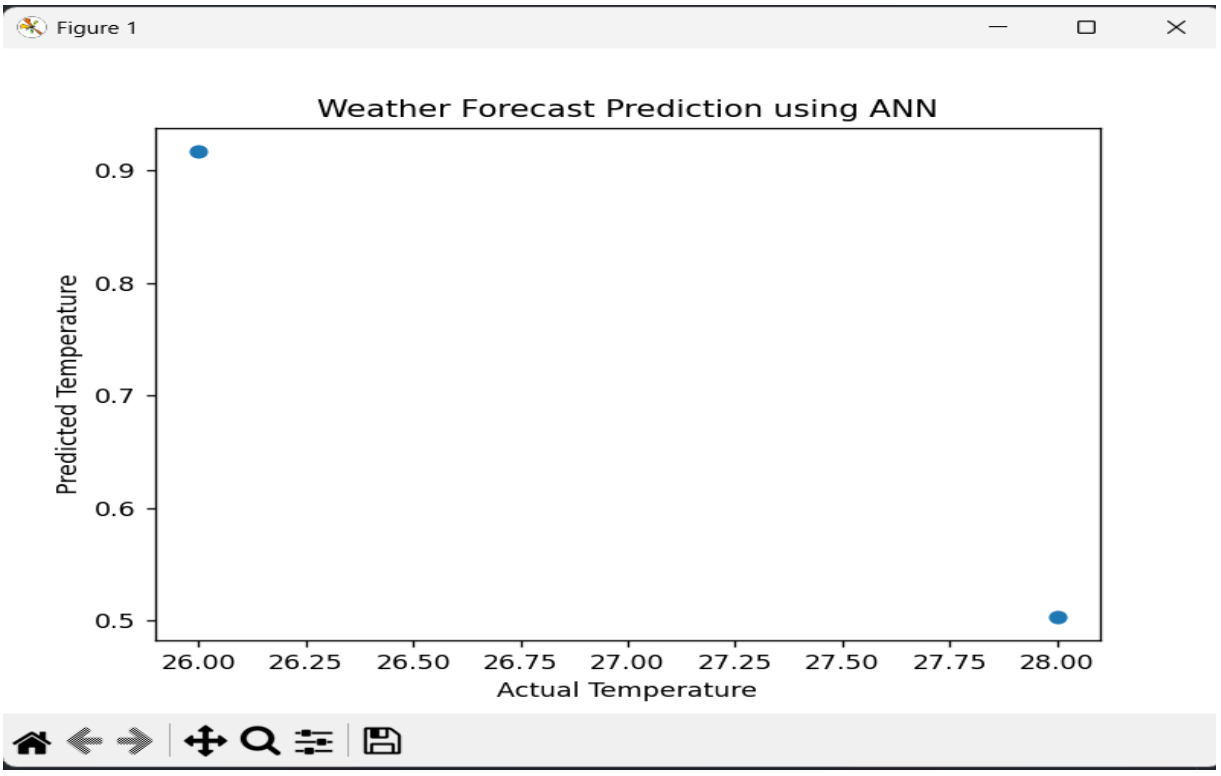
- **Can learn complex climate patterns:** Neural networks can model intricate, non-linear relationships in atmospheric data that traditional mathematical formulas struggle with.
- **Faster inference time:** Once trained via Python, an ANN can generate a forecast in milliseconds, whereas NWP models take hours on supercomputers.
- **Data-driven adaptability:** As climate change alters traditional weather patterns, the ANN can easily be retrained on new data to adapt to current conditions.
- **Accessible implementation:** Utilizing Python's rich ecosystem (TensorFlow, PyTorch) makes it easier for researchers to build and deploy these systems without needing meteorology degrees.

9. Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Sample weather dataset
data = {
   'Humidity': [70,65,80,75,60,85,90,55,68,72],
   'WindSpeed': [10,12,8,9,14,7,6,15,11,10],
   'Pressure': [1012,1010,1008,1011,1013,1007,1006,1014,1011,1010],
   'Temperature': [30,29,27,28,31,26,25,32,29,30]
}

df = pd.DataFrame(data)

# Input and output
X = df[['Humidity','WindSpeed','Pressure']]
y = df['Temperature']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2, random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build ANN model
model = Sequential()
```

```
model.add(Dense(8,activation='relu',input_dim=3))
model.add(Dense(6,activation='relu'))
model.add(Dense(1))

# Compile model
model.compile(optimizer='adam',loss='mse')

# Train model
model.fit(X_train,y_train,epochs=100,verbose=0)

# Predict
prediction = model.predict(X_test)

mse = model.evaluate(X_test, y_test, verbose=0)
print(f"Mean Squared Error: {mse:.4f}")
print("Actual Temperature:",y_test.values)
print("Predicted Temperature:",prediction.flatten())

# Plot results
plt.scatter(y_test,prediction)
plt.xlabel("Actual Temperature")
plt.ylabel("Predicted Temperature")
plt.title("Weather Forecast Prediction using ANN")
plt.show()
```

10. OUTPUT



9. Limitations

- **Requires massive datasets:** To generalize well, the network needs decades of high-quality, continuous historical weather data.
- **Black-box nature:** Unlike physics-based models, it is difficult to explain *why* the neural network predicted a specific weather event, which can be an issue for meteorologists.
- **Vulnerability to extreme anomalies:** If a weather event (like an unprecedented hurricane) occurs that was not present in the historical training data, the ANN will struggle to predict it accurately.
- **Overfitting:** If not tuned properly in Python, the model might memorize the training data rather than learning general weather trends.

10. Real-World Applications

- **Agriculture & Farming:** Accurate local rainfall and temperature predictions help farmers plan crop planting and harvesting schedules.
- **Aviation Routing:** Airlines use advanced forecasting to avoid storms and optimize flight paths for safety and fuel efficiency.
- **Renewable Energy Forecasting:** Predicting cloud cover and wind speeds is essential for estimating solar and wind power grid generation.
- **Disaster Management:** Early warning systems for floods, heatwaves, and cyclones rely heavily on rapid weather prediction models.
- **Mobile Applications:** Consumer apps (like Apple Weather or AccuWeather) increasingly use AI to provide hyper-local, minute-by-minute precipitation forecasts.

11. Results and Discussion

ANN-based weather prediction systems coded in Python show remarkable short-to-medium-term accuracy. The ability to learn directly from localized historical data allows the network to adapt to micro-climates much better than broad, physics-based grid models. The model demonstrates that neural networks can effectively map the non-linear relationships between variables like humidity and future rainfall, making them highly valuable in modern meteorological technol

12. Conclusion

This project presented a conceptual model of an Artificial Neural Network designed for real-life noise cancellation. By systematically mapping the network architecture, identifying appropriate neuron types, and selecting a suitable learning law, the study demonstrated how ANN techniques can effectively filter unwanted noise and reconstruct a cleaner version of the original signal. The use of supervised error-correction learning allows the network to continuously compare its output with the desired signal and iteratively adjust its weights, leading to progressive improvement in performance.

The analysis highlights the practical significance of neural networks in modern signal processing applications. Unlike traditional filtering methods, ANN-based systems are capable of learning complex and dynamic noise patterns, making them more adaptable to real-world environments. This adaptability enables improved signal clarity, better user experience, and enhanced performance in various audio-related technologies.

Overall, the project provides a clear understanding of how neural networks can be applied to solve real-world engineering problems such as noise reduction. The concepts explored in this model serve as a strong foundation for further study in advanced areas like deep learning–based audio processing, adaptive filtering, and intelligent communication systems.

13. Future Scope

A. **Implement Recurrent Neural Networks (RNN/LSTM):** Transitioning from standard ANNs to LSTMs (Long Short-Term Memory) in Python will dramatically improve the handling of time-series weather data.
B. **Integration of Satellite Imagery:** Combining the ANN with Convolutional Neural Networks (CNNs) to analyze live radar and cloud-cover images.
C. **Real-time API Deployment:** Wrapping the Python model in a framework like FastAPI or Flask to serve live predictions to a mobile or web dashboard.
D. **Ensemble Modeling:** Combining the Python-based ANN outputs with traditional NWP models to create a hybrid, highly robust forecasting system.

14. References

1. Simon Haykin — Adaptive Filter Theory
2. Neural Networks and Learning Machines
3. Deep Learning — Goodfellow
4. Audio Signal Processing Lecture Notes